

O uczeniu maszynowym i sieciach neuronowych

Paweł Rychlikowski

Instytut Informatyki UWr

14 maja 2026

Obok wnioskowania i przeszukiwania jeden z głównych **silników** sztucznej inteligencji.

Użyteczne między innymi w sytuacjach o których ostatnio mówiliśmy, gdy nie chcemy **pamiętać** wartości $Q(s, a)$ lecz umieć ją **obliczyć**

Uwaga

Przedmiot uczenie maszynowe – będziemy starali się unikać przecięcia materiałów (pominąwszy konieczne powtórki).

Psy i koty. Konkurs na Kaggle.com

In this competition, you'll write an algorithm to classify whether images contain either a dog or a cat. This is easy for humans, dogs, and cats. Your computer will find it a bit more difficult.



Deep Blue beat Kasparov at chess in 1997.
Watson beat the brightest trivia minds at Jeopardy in 2011.
Can you tell Fido from Mittens in 2013?

Uczenie z nadzorem

Dane uczące



Uczenie z nadzorem

Dane uczące i dane testowe



?



?



?



?



?

- Spróbujemy usystematyzować nasze intuicje związane z uczeniem.
- Co wiemy:
 - a. Mamy przykłady, próbujemy je **uogólnić**.
 - b. Jedno z podstawowych zadań: **klasyfikacja**, czyli przypisanie **przypadkowi** jego **klasy**.
 - c. Przykłady:
 - Ocena, czy **mail** należy do **spam** czy też **nie-spam**.
 - Wybór **rasy** dla **zdjęcia psa**
 - Czy **napis** jest **adresem e-mail**, **url-em**, **nazwą firmy**, **imieniem i nazwiskiem**, **czymś innym**?

- Oczekiwanym wynikiem może być liczba rzeczywista.
- Przykłady:
 - predycja ceny nieruchomości,
 - ocena masy ciała (gdy znamy płeć i wzrost),
 - przewidywanie zużycia wody (dla MPWiK), gdy znamy temperaturę i dzień tygodnia

W klasyfikacji i regresji inaczej oceniamy sukces, w regresji **liczba** nie musi być dokładna, wystarczy, że jest blisko.

Uwaga

Zadanie rozróżnienia kota i psa byłoby łatwiejsze, gdybyśmy mieli dane nie obrazki, lecz cechy zwierzęcia

Przykłady?

- masa ciała,
- odległość między oczami,
- odległość nosa i oka,
- długość włosów,
- długość wąsów,
- długość ogona

użyteczne mogłyby być też na przykład proporcje różnych cech i inne **wtórne cechy** (wyliczone z podstawowych)

Cechy (wektor cech)

- Abstrakcyjny **obiekt** możemy zamienić na **wektor cech**.
- Dla (zabawkowego) klasyfikatora **czy-email?**, możemy mieć:
 - Czy długość większa od 10?
 - Jaki procent znaków to znaki alfanumeryczne?
 - Czy zawiera @?
 - Czy kończy się na **.com** (i tak dalej)

Cechami mogą być też na przykład wartości składowych pikseli, kolejne wartości pliku wave, zbiory pomiarów wszystkich wodomierzy z ostatniej doby, itd.

Dla obiektu x wektor cech oznaczamy często jako $\phi = (\phi_1(x), \dots, \phi_n(x))$, $\phi \in \mathcal{R}^n$.

- Dane uczące – zbiór przykładów, często w postaci:
 (*wektor-cech1*, *wynik1*)
 (*wektor-cech2*, *wynik2*)
 (*wektor-cech3*, *wynik3*)
 ...

Podział dostępnych danych

Definicja 1

Zbiór uczący jest podstawowym zbiorem, który będziemy wykorzystywać do zdobywania wiedzy o problemie.

Definicja 2

Zbiór **walidacyjny** używamy do wyboru rodzaju algorytmu lub do wyboru hiperparametrów algorytmu.

Definicja 3

Zbiór testowy używany jest **tylko** do ostatecznego testu, który ma nas przekonać, jak dobrze uogólnia rzeczywistość nasz mechanizm.

Do zbioru testowego nawet nie zaglądamy, nie analizujemy błędów, itd!

Klasyczne zadanie klasyfikacji obrazów

MNIST jest zbiorem około 60K czarnobiałych obrazków 28×28 zawierających ręcznie pisane cyfry.

Jest on powszechnie używany do testowania różnych algorytmów uczenia (głównie klasyfikacji, ale nie tylko)

MNIST – przedstawienie danych



MNIST-style datasets

A



B



A: EMNIST, B: Kuzushiji-MNIST, C: Fashion-MNIST

Kolorowy odpowiednik: CIFAR-10

airplane



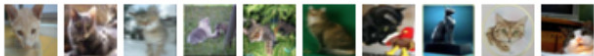
automobile



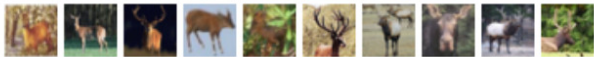
bird



cat



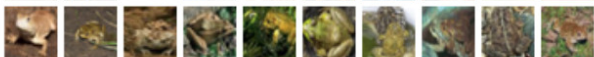
deer



dog



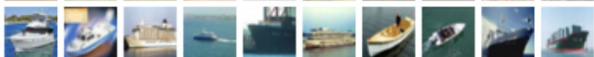
frog



horse



ship



truck



- Naturalnym rozwiązaniem jest stworzenie **wzorca** (lub wzorców) dla każdej cyfry.
- Jak to zrobić?

Dwa warianty

1. Jeden wzorec dla wszystkich obrazków danej cyfry.
2. Wiele wzorców (nawet: **każda cyfra wzorcem**)

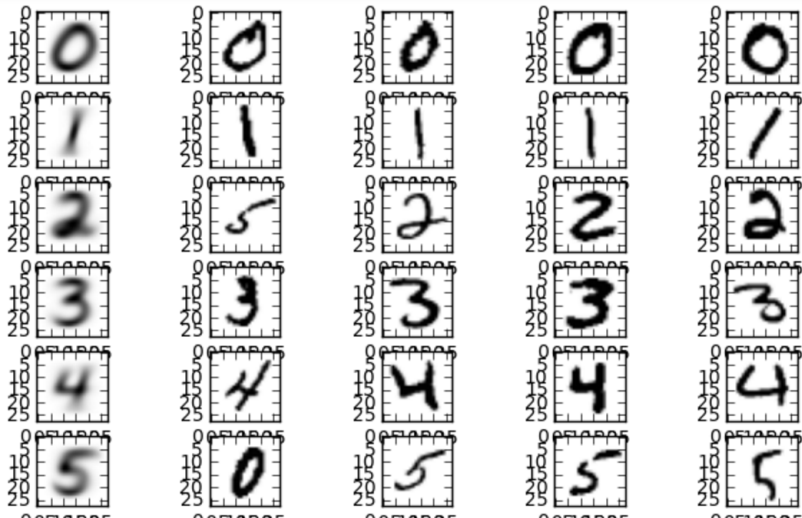
Jeden wzorzec dla cyfry

- Naturalnym wzorcem może być średnia wszystkich egzemplarzy danej cyfry
- Przy klasyfikacji obrazka $\mathbf{o}$ wybieramy wzorzec $\mathbf{w}$ **najbardziej podobny** do $\mathbf{o}$
- Co to znaczy podobny?
 - Wysoki iloczyn skalarny?
 - *Wysoki iloczyn skalarny znormalizowanych wektorów?*
 - Niewielka odległość euklidesowa (a może jakaś inna)?

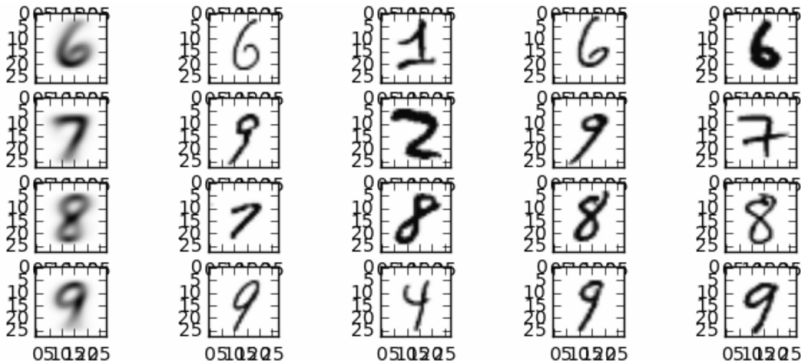
Wyniki eksperymentu

Poprawność klasyfikacji to około **82.1%** (dla iloczynów skalarnych znormalizowanych wektorów)

Wyniki klasyfikacji z „wzorcami średnimi”

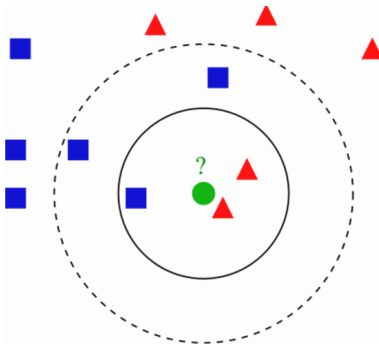


Wyniki klasyfikacji z „wzorcami średnimi”



K najbliższych sąsiadów. KNN

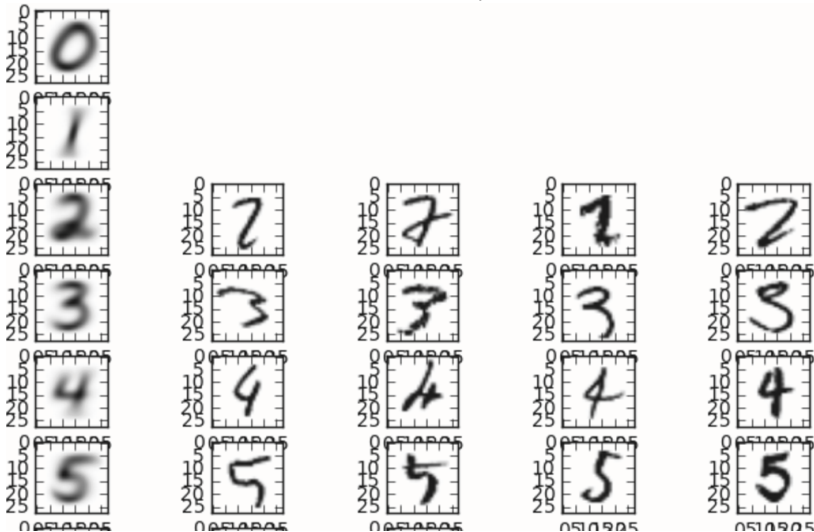
- Pamiętamy wszystkie wektory ze zbioru uczącego.
- Klasyfikując obrazek znajdujemy K najbliższych sąsiadów i pozwalamy im głosować



- Podobieństwo mierzymy iloczynem skalarnym znormalizowanych wektorów (czyli **cosinusem**)
- Testujemy na próbce (bo inaczej trwa bardzo długo)
- $K = 3$
- Wyniki:
 - Zbiór uczący: **98.55%**
 - Zbiór testowy: **około 97%**

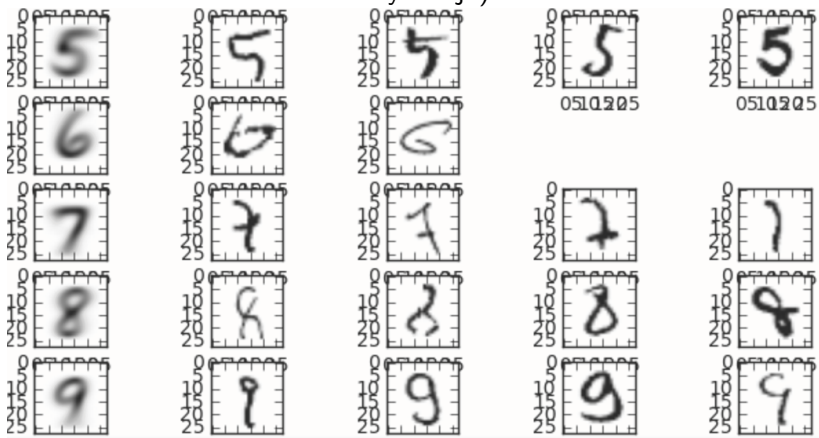
MNIST i KNN. Przykładowe błędy

Z tymi cyframi mieliśmy problemy (zaznaczona prawidłowa klasyfikacja)

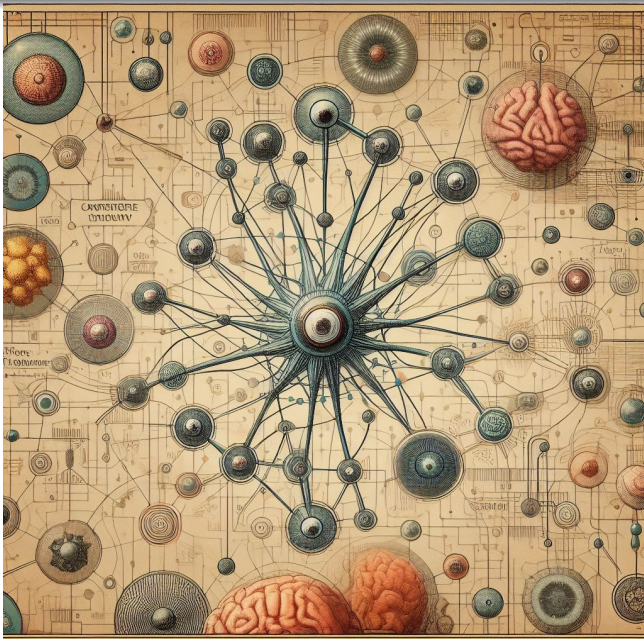


MNIST i KNN. Przykładowe błędy

Z tymi cyframi mieliśmy problemy (zaznaczona prawidłowa klasyfikacja)



Sieci neuronowe



Sieci neuronowe w paru prostych slajdach

- Wybierzemy absolutne minimum tego, co należy wiedzieć o sieciach neuronowych
- Oczywiście nie będzie to w pełni kompletna wiedza.

- Neuron to funkcja $f : \mathcal{R}^n \rightarrow \mathcal{R}$

$$f(x_1 \dots x_n) = \sigma\left(\sum_{i=1}^n w_i x_i + b\right)$$

- σ jest jakąś ustaloną funkcją nieliniową, raczej rosnącą, raczej różniczkowalną, na przykład: $\max(0, v)$, albo $\tanh(v)$
- Wygodna (jak za chwilę zobaczymy) jest notacja wektorowo-macierzowa, w niej mamy:

$$f(x) = \sigma(w^T \cdot x + b)$$

Slajd 2. Prosta sieć neuronowa

- Warstwa to funkcja $\mathcal{R}^n \rightarrow \mathcal{R}^m$.
- Najbardziej typowa warstwa wyraża się wzorem:

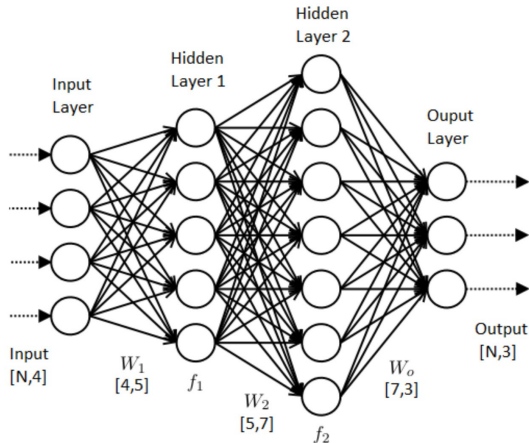
$$L(x) = \sigma(Wx + b)$$

- **Uwaga:** W jest macierzą wag (złożoną z wektorów wag), a $\sigma(y_1 \dots y_m) = (\sigma(y_1) \dots \sigma(y_m))$

Definicja

Sieć neuronowa typu **MLP** jest złożeniem warstw (z różnymi macierzami wag dla każdej warstwy).

Slajd 2b. Prosta sieć neuronowa



Źródło: VIASAT (<https://medium.com/coinmonks/the-artificial-neural-networks-handbook-part-1-f9ceb0e376b4>)

Slajd 3. Uczenie sieci

Zadanie

Danymi jest ciąg (x_i, y_i) opisujący porządane zachowanie sieci S oraz architektura tejże sieci (liczba warstw, ich wymiary, funkcja/funkcje σ).

Chcemy tak dobrać parametry (W_k oraz b_k) żeby dla każdego i

$$S(x_i) \approx y_i$$

Funkcja kosztu

Powyższe zadanie formalizujemy jako zadanie znalezienia takich parametrów, że **koszt** błędów jest jak najmniejszy. Przykładowo, jeżeli wyjściem jest liczba, to możemy wybrać:

$$\text{Loss}(\theta) = \sum_i^n (S_{\theta}(x_i) - y_i)^2$$

Uczenie jako optymalizacja

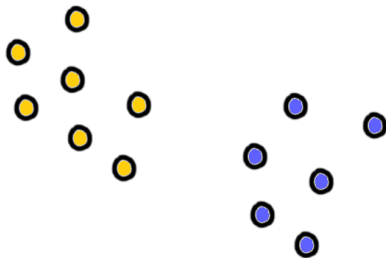
- Szukamy takich parametrów θ , że koszt jest najmniejszy
- W zasadzie moglibyśmy użyć dowolnego znanego algorytmu (Hill Climbing, Wyżarzanie, Beamsearch, algorytmy ewolucyjne)
- Algorytmy ewolucyjne to nawet były używane w tej roli, bo mogą optymalizować nie tylko wagi, ale również architekturę sieci

Ale niekwestionowanym liderem są tu **metody gradientowe**, które korzystają z tego, że optymalizowana funkcja jest różniczkowalna (o gradientie za chwilę)

Ogólne założenia (przypadek dwóch klas)

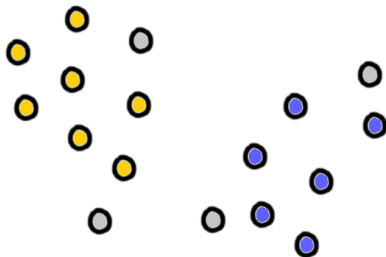
Mamy jakiś zbiór przykładów **pozytywnych** i **negatywnych**, interesuje nas mechanizm, który będzie poprawnie klasyfikował nieznane przykłady.

Klasyfikacja w $\mathcal{R}^2$



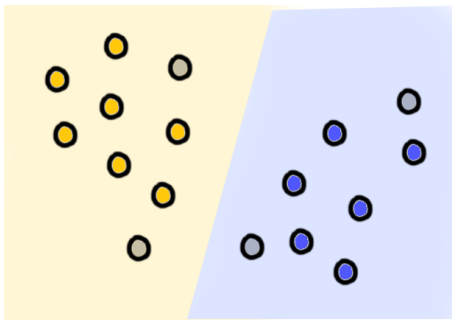
- Dane punkty wraz z informacją o kolorze.
- Mechanizm powinien umieć określać kolor nieznanych punktów.
- Możemy o tym myśleć, jako o „kolorowaniu płaszczyzny”

Klasyfikacja w $\mathcal{R}^2$



- Dane punkty wraz z informacją o kolorze.
- Mechanizm powinien umieć określać kolor nieznanych punktów.
- Możemy o tym myśleć, jako o „kolorowaniu płaszczyzny”

Klasyfikacja w $\mathcal{R}^2$



- Dane punkty wraz z informacją o kolorze.
- Mechanizm powinien umieć określać kolor nieznanych punktów.
- Możemy o tym myśleć, jako o „kolorowaniu płaszczyzny”

Spróbujmy poeksperymentować chwilę z Tensorflow Playground

Spróbujmy poeksperymentować chwilę z Tensorflow Playground